

YOURSTORY



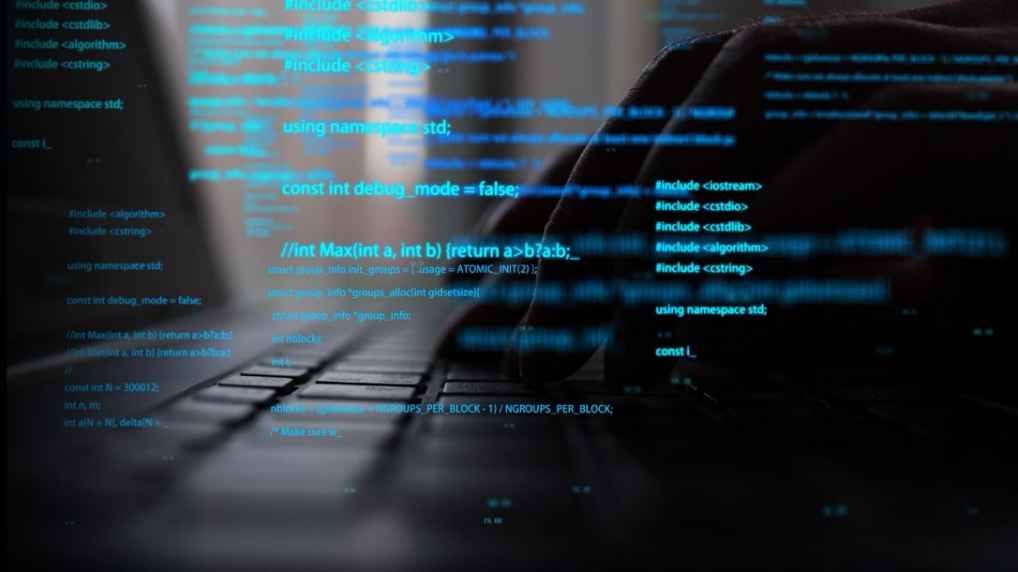
I12S



Coco CLI

HACKATHON

GCC Edition



Team Name : SupplyGraph AI

Team Leader Name : Parv Gour

Team Size : 1 (Individual Contributor)

Problem Statement : Supply-chain data is fragmented and inconsistent across teams, so the same question can produce different answers.

1. Problem Brief

One question. Multiple systems. Different definitions. SupplyGraph AI creates one governed answer.

REAL BUSINESS PROBLEM

Supply-chain data is spread across ERP, logistics (TMS), supplier and IoT systems, each with its own metric definitions. On the **same 6M shipment lines**, on-time delivery is **49.6%** (ERP), **38.4%** (logistics) and **100%** (supplier portal), versus **36.79%** governed. The same question gets a different answer from every team.

CURRENT PAIN POINT → HOW IT IMPROVES

Today: each team writes its own SQL and definitions, meetings turn into reconciliation, and AI chat adds another ungoverned interpretation.

With SupplyGraph AI: a Snowflake Semantic View defines each entity, relationship and metric once. Cortex Analyst answers plain-English questions from it, showing SQL, definition and source.

TARGET USERS / PERSONAS

Planning: demand, lead time, days of inventory.

Procurement: supplier performance, spend, landed cost.

Logistics: ship modes, lanes, IoT shipment risk.

Each persona asks in its own words; the semantic layer resolves every phrasing to the same governed metric.

INDUSTRY / DOMAIN CONTEXT

Manufacturing and distribution supply chains: Supplier → Part → Plant → Shipment → Order → Customer. KPIs: on-time delivery, fill rate, days of inventory, landed cost, supplier performance, IoT cold-chain risk. Built on Snowflake TPC-H (6M shipment lines, 10K suppliers) plus clearly labelled synthetic ERP, logistics and IoT enrichment.

2. Architecture Diagram

Source data → curated views → governed ontology (Semantic View) → Cortex Analyst → Streamlit in Snowflake. 100% Snowflake-native.



COCO CLI SKILLS / CAPABILITIES

- **Environment discovery:** roles, warehouse, data, Cortex models
- **cortex search docs:** Semantic View DDL research
- **SQL execution:** views, enrichment, semantic view
- **cortex analyst query:** 22 NL questions tested; CTE failures diagnosed (6/15 → 22/22)
- **Streamlit in Snowflake skill:** app build and deploy
- **Python REPL + AppTest:** headless end-to-end tests
- **git:** versioned GitHub release

DATA SOURCES

Structured, original: TPC-H SF1: LINEITEM, ORDERS, PARTSUPP, PART, CUSTOMER, SUPPLIER, NATION, REGION.

Structured, SYNTHETIC (SYN_): ERP plant master (10 plants), freight tariff, customs duty tariff, IoT telemetry (6M rows; delay alert from real lateness).

Metadata: per-system metric definitions (ERP / TMS / supplier portal / governed).
No unstructured data.

HOW MODULES PLUG TOGETHER

- Every KPI, chart and persona view is a **SEMANTIC_VIEW()** query; no metric logic in app code.
- Chat answers come from **Cortex Analyst** on the same view, with SQL and definitions shown.
- Plants, landed cost and IoT were added as new logical tables **without changing existing metrics**.
- If Analyst is unreachable, a **verified query** is used, never an invented number.

3. Impact Statement

One definition per KPI, one semantic source of truth, the same answer for every persona.

22/22

Natural-language questions answered correctly by Cortex Analyst

18/18

Governed metrics match independent SQL on raw sources exactly

12/12

Rephrasings return identical governed rows (wording consistency)

3 → 1

Personas get one identical answer (Planning, Procurement, Logistics)

MEASURABLE OUTCOMES

- **Definition conflict removed:** on-time delivery ranged from 36.79% to 100% across systems; now one governed value everywhere.
- **Accuracy:** Cortex Analyst raised from **6/15 to 22/22** by root-causing its SQL generation and hardening the ontology.
- **Trust:** every answer shows SQL, definition and source; 18 metrics validated by a reproducible script.
- **Zero data movement:** zero-copy views; no new infrastructure.
- **Robustness:** 14/14 headless app scenarios, incl. outage fallback and million-row answers capped safely.

SCALABILITY & BEYOND THE DEMO

- **Scales with Snowflake:** 6M-row fact tables queried live; the same model runs on larger or production data.
- **Real sources:** swap SYN_ tables for live ERP, TMS and IoT feeds (Openflow, Snowpipe Streaming) without changing the ontology.
- **More domains:** add warehouses, carriers or forecasts as logical tables; existing metrics stay stable.
- **More consumers:** the same semantic view can serve Cortex Agents, BI tools and APIs.
- **Access governance next:** row-access and masking policies per persona.

4. Challenge Coverage & Evidence

Every Challenge 5 requirement mapped to a working, validated implementation, with explicit data provenance.

Ontology: entities, relationships, hierarchies	12 logical tables, 11 relationships; Supplier → Part → Plant → Shipment → Order → Customer; geography, plant, product and time hierarchies
Canonical metrics	On-time delivery 36.79% · fill rate 75.36% · days of inventory (TPC-H) · landed cost \$80.02B (TPC-H + SYNTHETIC tariffs); 39 metrics in total
Encoded as Semantic Views	SUPPLY_CHAIN_ONTOLOGY with synonyms, comments, AI_SQL_GENERATION rules and 21 verified queries
Governed conversational analytics	Cortex Analyst: 22/22 incl. cross-domain (landed cost × IoT × plant); 'IoT risk' is a governed term (12/12 identical)
Same metric across personas	Live hero demo: three phrasings → IDENTICAL; same governed KPI fingerprint for all personas
Evidence & traceability	SQL, definition and source shown per answer; sql/07_validation.sql: 18/18 PASS vs raw tables

Honest limitations: plants, freight and duty tariffs, IoT telemetry and the per-system definitions are **synthetic** and labelled (no live connectors). Days of inventory is genuinely computed but very high, because TPC-H availability is not calibrated to demand. Fill rate is a proxy (lines not returned). Access policies (RBAC, masking) are not yet implemented.

YOURSTORY



1125

COCO CLI

HACKATHON

GCC Edition

THANK YOU

GitHub: github.com/Parvgour98/supplygraph-ai · App: SUPPLYGRAPH_AI.SUPPLY_CHAIN.SUPPLYGRAPH_AI_APP (Streamlit in Snowflake)